Homework 3 Summary

Logan Clevenger

CPSC – 224 – 02

10/29/2026

****Disclaimer**** - ChatGPT was used to assist in UML Class and Sequence Diagram Planning and Organization. It was also used to learn more and gain a deeper understanding when I was confused regarding a relationship/process.

Goal/Purpose of Program:

In this final phase of the Farkle game development, the goal was to add to my existing implementation that only handled a single round for a single player and expand upon this to include multiplayer and be able to reach an endgame state with a winner. Regarding scoring, the Full House combo was added, which allows players to receive 1500 points for banking 5 dice in their meld where there's both a triple and a double of separate die values. Additionally, functionality was added to allow the players of Farkle to choose the number of points required to reach endgame (and the default for this setting is 10000). To add to the multiplayer functionality, players are also now able to enter the number of players they would like to play with, and names are received from each individual player (default names include Pumpkin, Zombie, Ghost, Vampire); finally, each player has their own individual points that are tracked. After each player finishes a turn, all scores are displayed to show the current game standings. An endgame phase has been added, in which once a player reaches the required endgame points, every other player gets a final turn to bank as many points as possible (and possibly still win!). At the end of this endgame rotation, a winner screen is displayed, and it is Halloween-themed to be festive.

From this, we've reached the overall goal of complete Zag-Farkle implementation!

Overview of General Program Design:

For this final project implementation, I've utilized 6 different classes. I first made a Die class that represents one die in the Farkle game. This class stores its current numeric value and is able to roll randomly to another value (between 1-6). I then made a Hand class that owns 6 die instances. This class also tracks selections being made between the player's hand and their meld, along with whether or not the player rolled a Farkle. Following that, I made a Meld class. This class stores selected dice from the player's hand and calculates the Zag Farkle score of the dice currently contained in the player's meld (Full House combo scoring was added in this implementation). Next, I made the Farkle class. This class actually runs the game and controls menu display and user input for die selections. In homework 3, the Farkle class was further

expanded to handle full game completion management, multiplayer (and all info attached to players), endgame rotation, and winner celebration and display. I also have added the Player class. This class allows for storage of the current player's name, distinct player ID, and the banked points attached to this player. Finally, I made a Scoreboard class which stores all players participating in the game, manages the active player, displays the current scores of every player, detects when a player has reached the winning amount of points, and manages endgame rotation along with identifying the winner of Zag-Farkle.

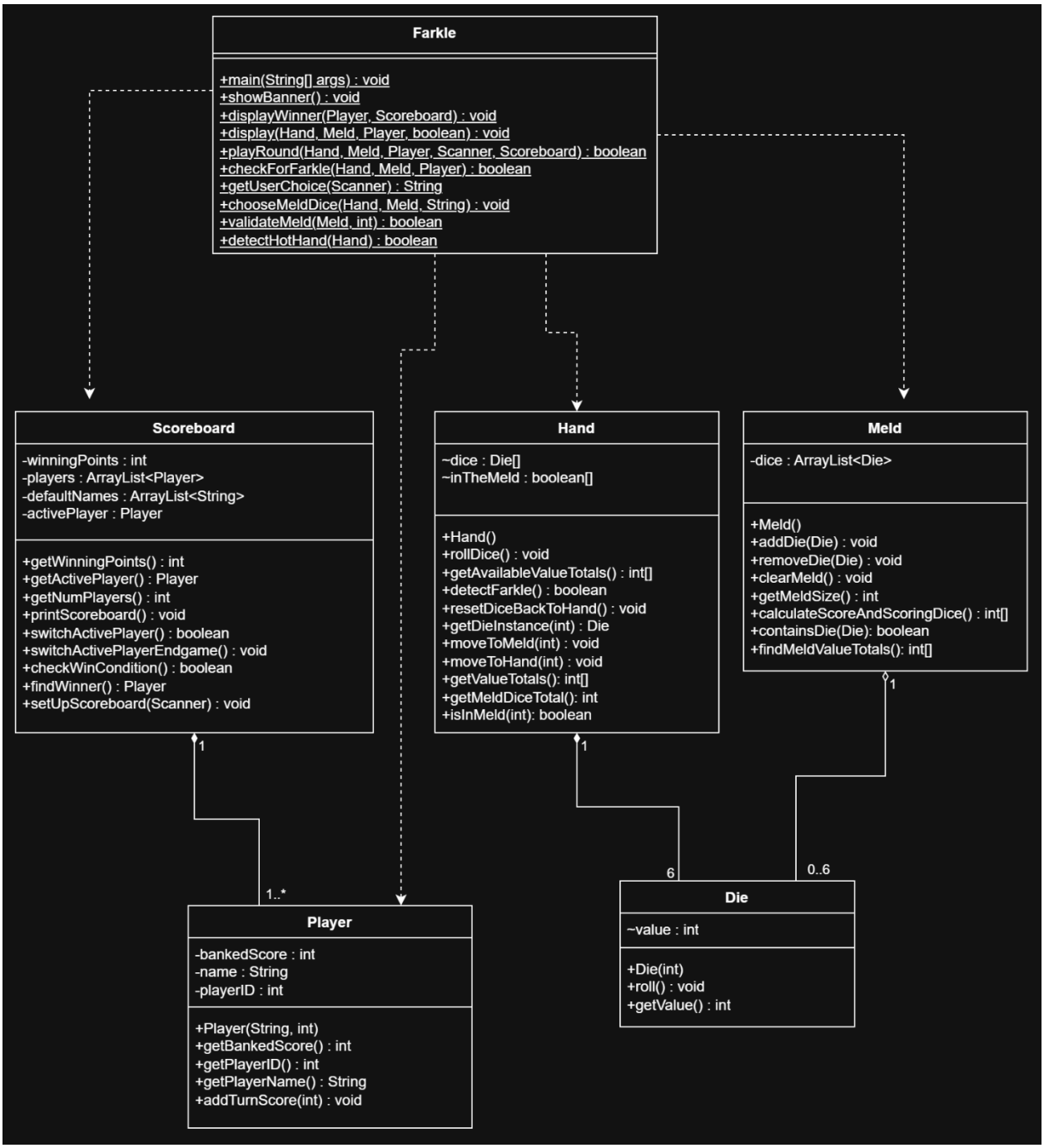
Description of Unit Test:

The unit test I've chosen to describe in depth is my "testFullHouseExtraOne()" test.

```
114
115     /**
116      * Tests Full House (with 1's) with an additional 1 value die
117      */
118     @Test
119     public void testFullHouseExtraOne() {
120         Meld meld = new Meld();
121
122         meld.addDie(new Die(value: 1));
123         meld.addDie(new Die(value: 1));
124         meld.addDie(new Die(value: 1));
125         meld.addDie(new Die(value: 2));
126         meld.addDie(new Die(value: 2));
127         meld.addDie(new Die(value: 1));
128
129         int [] results = meld.calculateScoreAndScoringDice();
130
131         assertEquals(1600, results[0]);
132         assertEquals(6, results[1]);
133     }
```

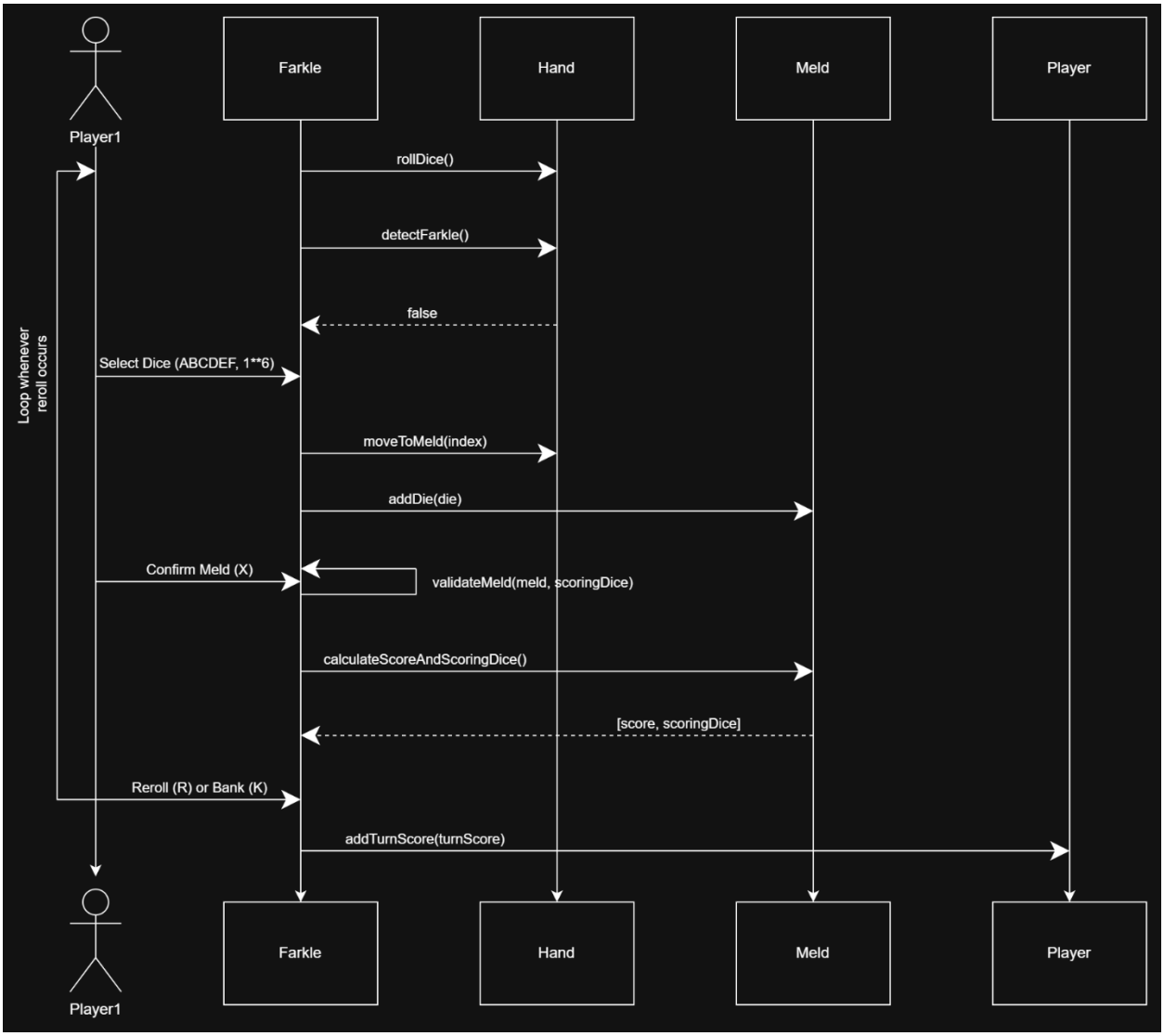
In this test, the meld is set to contain die values of 1,1,1,2,2,1. With the new Full House combo being implemented, the triple 1's and the double 2's should be counted as a Full House, and we award 1500 points. The triple 1's should also not be counted for traditional triple scoring. Additionally, in this test, the meld contains an extra 1 beyond the triple 1s. This tests an edge case as the full house must be recognized, and the additional 1 should not be looped in with the 1's already counted in the Full House and instead should count as 100 additional points (as individual 1's are normally scored). As such, the final number of scoring dice should be 6, and the value of the awarded points to the player should be 1600. This is verified through the final 2 assert statements, which reference the integer array "results" returned by the meld.calculateScoreAndScoringDice(). Results is organized as {awarded points, scoring dice}.

UML Class Diagram:



UML Sequence Diagram:

This diagram demonstrates the turn behavior of a player rolling a die and determining a valid roll or a Farkle, moving the die back and forth between the Hand and the Meld, confirming Meld and validating the Meld, choosing to bank or reroll if applicable, and finally adding their turn score to their personal score.



Major Design and Programming Issues and Solutions:

The first design issue I ran into was handling how I could create preset defaults for player names while still allowing players access to choose any number (positive) of players that they would like to participate in the game. To handle this, I settled on including 4 default names for players, as I estimate this would be the most common max amount of players playing hotseat on a single computer. These first 4 default names are “Pumpkin”, “Zombie”, “Ghost”, “Vampire” to stick with my Halloween theme on this implementation, and they will be assigned when a player does not enter a name. If more than 4 players are playing hotseat and my preset default names are used up, any additional players will be named “Unknown Player #{player_num}” if they don’t enter a name for themselves.

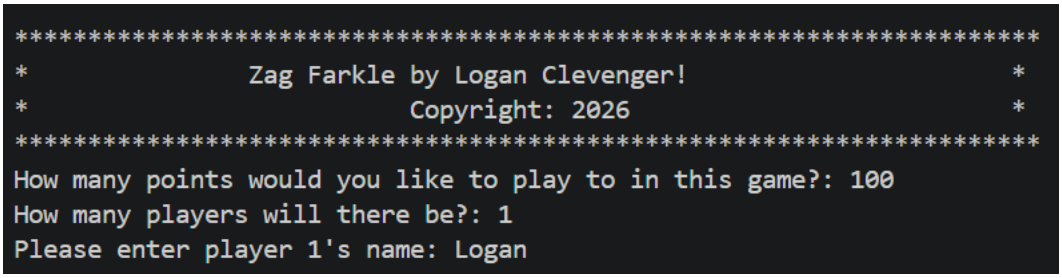
The next design issue I had was handling the endgame rotation. In the beginning, I was confused as I had added game condition validation inside my original switch player function within Scoreboard. As the game win condition is known to have already been reached in endgame rotation, trying to use this functionality again would raise issues. As such, I decided to just use a for loop that rotates through the players array list stored in Scoreboard, excluding the player who reached the end condition first. As such, I just added an additional method in Scoreboard, .switchActivePlayerEndgame() that does not use any “win” validation and just rotates to the next available player to solve my issue.

Retrospective:

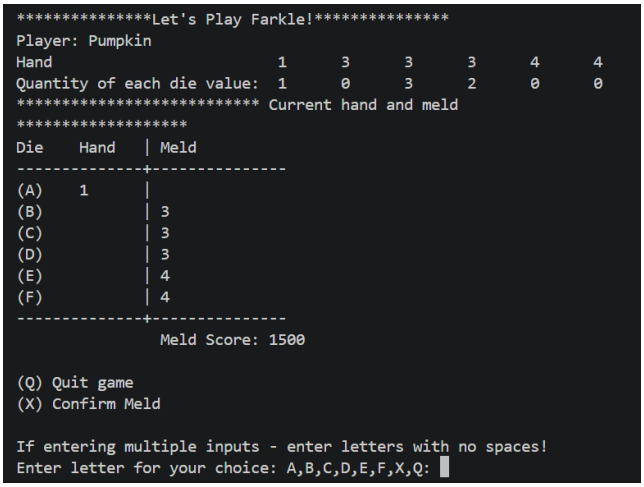
In retrospect, I am satisfied with my finalized Zag-Farkle implementation. However, if I had more time, I would spend more time creating a more aesthetic game flow. Especially when one player rolls a Farkle, the transition between players can be a bit jarring and confusing for the overall gameplay. In that regard, I could potentially have an option for players to acknowledge that they rolled a Farkle (even just pressing enter) to allow them to recognize that their turn immediately resulted in a Farkle and will then switch to the next player in line. But overall, I believe that my Zag-Farkle game is well constructed and fun! I’m also glad that I got to include a Halloween-themed results banner!

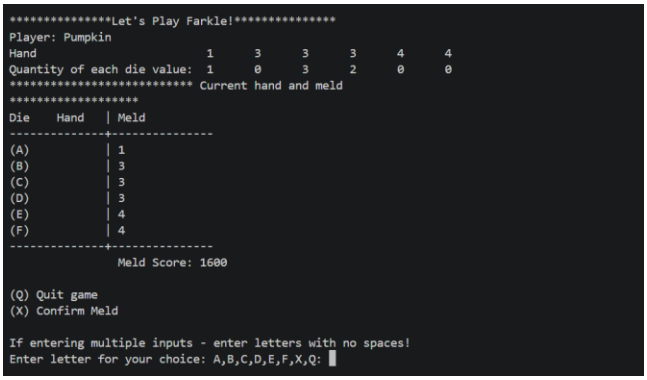
Appendix

Intro Screen Screenshot



Full House Combo Screenshots





Multiplayer Game Screenshots (entering players, total scores, winner's screen)





